



## INFORME DEL TRABAJO PRACTICO

### I. PORTADA

|                                    |                                            |
|------------------------------------|--------------------------------------------|
| Tema:                              | SW-AyLP-APE-4.0: Clases, objetos y métodos |
| Unidad de Organización Curricular: | BÁSICA                                     |
| Nivel y Paralelo:                  | Primero Software “B”                       |
| Alumnos participantes:             | Sánchez Lema Isaac Adrián                  |
|                                    | .....                                      |
| Asignatura:                        | ALGORITMOS Y LÓGICA DE PROGRAMACIÓN        |
| Docente:                           | Ing. José Caiza, Mg.                       |

**SW-AYLP-APE-4.0: Clases, objetos y métodos.**

### II. INFORME DEL TRABAJO PRACTICO

#### 1.1 Objetivos

##### General:

- Desarrollar un sistema informático de gestión académica en C++ / Java, implementando estructuras de datos dinámicas y algoritmos de control lógico, para optimizar el registro continuo de calificaciones, la evaluación del estado estudiantil y la generación automática de reportes estadísticos.

##### Específicos:

- **Diseñar e implementar una clase robusta para la validación de entrada de datos**, garantizando la integridad y veracidad de la información ingresada mediante la ejecución de algoritmos como el Módulo 10 para el identificador y el control estricto de límites numéricos para las notas.
- **Aplicar estructuras de datos eficientes** (mediante el uso de arreglos paralelos o *structs* orientados a objetos) para asegurar el almacenamiento temporal estructurado y la correcta manipulación de los expedientes de cada estudiante durante el ciclo de ejecución del programa.
- **Automatizar el cálculo de métricas de rendimiento académico**, procesando los promedios individuales para determinar la aprobación o reprobación del semestre, y asegurando la persistencia de este reporte estadístico final a través de la exportación a un archivo físico (CSV).

#### 1.2 Modalidad

Presencial.

#### 1.3 Tiempo de duración

**Presenciales: 6.**

**No presenciales: 0.**

#### 1.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++ además de Java.
- Subir en un documento en formato PDF que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código



generado, y subirlo al aula virtual.

- Si se manifiestan dudas se invita a los estudiantes a acudir a las tutorías académicas.

### 1.5 Listado de equipos, materiales y recursos

- Inteligencia artificial, TAC Inteligencia artificial, TAC Inteligencia artificial, TAC.
- - Computador o laptop.
- - Plataforma educativa de la UTA.
- - Internet para consultas. - Papel - Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): \_\_\_\_\_

### 1.6 Actividades por desarrollar

- Desarrollar la guía APE en base al formato y las instrucciones brindadas.

### 1.7 Resultados obtenidos

- Se logró implementar con éxito una clase de validación de entrada de datos que filtra eficientemente la información ingresada por el usuario. El sistema demostró ser capaz de rechazar identificadores inválidos mediante la correcta aplicación del algoritmo Módulo 10, y bloqueó eficazmente el ingreso de calificaciones numéricas fuera del rango permitido (0.00 a 10.00). Esto redujo a cero el margen de error humano por tipografía durante las pruebas de registro de notas.
- La aplicación de estructuras de datos (arreglos paralelos y manejo de *structs*) permitió una gestión de memoria fluida y sin colisiones. Durante la ejecución del programa, el sistema fue capaz de almacenar, enlazar y consultar los datos personales (nombres, cédula) junto con los datos académicos (notas, promedio, estado) de múltiples alumnos de forma simultánea, manteniendo la integridad del expediente de cada estudiante durante todo el ciclo operativo.
- Se obtuvo un sistema capaz de procesar la lógica de negocio de manera automática e instantánea. El algoritmo calculó con exactitud los promedios aritméticos y clasificó correctamente a los estudiantes en los estados de "APROBADO" o "REPROBADO" según el umbral de 7.0. Además, se logró concretar la persistencia física de los datos; el programa exportó satisfactoriamente el listado completo y las métricas estadísticas totales a un archivo externo en formato .csv, garantizando que la información académica quede respaldada para futuras consultas.

### Resultado General del Proyecto

- Como resultado global, se entregó un prototipo funcional de software por consola que cumple a cabalidad con los requerimientos lógicos y matemáticos establecidos, brindando una herramienta estructurada y eficiente para el control y seguimiento del estado académico semestral.

### 1.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo



- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☒ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

### 1.9 Conclusiones

- Se concluye que el diseño e implementación de una clase especializada de entrada de datos es un pilar fundamental para la seguridad del software. La integración del algoritmo Módulo 10 y la restricción estricta de rangos para las notas demostraron que es posible mitigar por completo el ingreso de datos corruptos o erróneos, garantizando la consistencia de la información antes de que sea procesada por las capas lógicas del sistema.
- La aplicación práctica de arreglos paralelos y estructuras de tipo *struct* confirmó que la organización eficiente de la memoria RAM es clave para el rendimiento de una aplicación. Se determinó que mapear entidades del mundo real (como un estudiante y sus calificaciones) en bloques de datos enlazados permite un acceso ágil y ordenado, facilitando la escalabilidad del código y el mantenimiento del estado del programa durante su ejecución.
- La automatización de las reglas de negocio, mediante el cálculo inmediato de promedios y la clasificación de estados académicos, valida la utilidad de la programación estructurada para optimizar tareas administrativas propensas al error manual. Asimismo, la correcta exportación de estas métricas hacia archivos planos (.csv) concluye que el software cumple de forma óptima con el ciclo de vida del manejo de datos: captura, procesamiento, análisis y almacenamiento permanente.

### Vinculación con el proyecto final de la asignatura

- Finalmente, el desarrollo y desglose de este sistema académico (desde su lógica algorítmica en PSeInt hasta su codificación limpia en C++ y Java) constituye la base arquitectónica y pedagógica para la creación del aplicativo final de enseñanza de programación. Al haber aislado con éxito los componentes críticos del código —como los bucles de control, la validación modular y las estructuras de datos simples— se ha definido el mapa de ruta ideal para los estudiantes de primer semestre. Este núcleo lógico servirá como el primer gran módulo interactivo dentro de la app educativa, permitiendo a los nuevos alumnos visualizar de forma intuitiva cómo la lógica abstracta del pseudocódigo se transforma paso a paso en software funcional y profesional dentro de la ingeniería de software.

### 1.10 Recomendaciones

- Se recomienda encapsular de forma definitiva la clase de validación y modularizar el código para futuras expansiones. Esto permitirá reutilizar las funciones de verificación de texto, límites numéricos y el algoritmo Módulo 10 en otros sistemas de la institución, reduciendo los tiempos de desarrollo y asegurando un estándar de calidad en la captura de información.
- Para optimizar el manejo de memoria en escenarios con una mayor carga de datos, se sugiere realizar la transición de arreglos estáticos o paralelos hacia colecciones dinámicas nativas de la programación orientada a objetos (como `std::vector` en C++ o `ArrayList` en Java). Esto evitará la limitación de un tamaño fijo de almacenamiento y prevendrá



posibles desbordamientos de memoria (*buffer overflow*).

- Se recomienda enriquecer el módulo de reportes implementando estructuras de control que calculen medidas estadísticas avanzadas (como la nota más alta, la más baja y la desviación estándar). Asimismo, para la persistencia de datos, se aconseja migrar el almacenamiento de archivos .csv hacia un sistema de gestión de bases de datos relacionales (como SQLite o MySQL) para garantizar la seguridad y concurrencia de la información.

### **Vinculación con el proyecto final de la asignatura**

- Con miras al desarrollo del aplicativo final para la enseñanza de programación, se recomienda **utilizar la arquitectura modular de este proyecto como el eje didáctico principal del software educativo**. Se sugiere diseñar la interfaz del aplicativo de modo que el estudiante de primer semestre pueda interactuar con este algoritmo dividiéndolo en tres fases visuales: primero, un área interactiva para entender las validaciones básicas; segundo, un simulador de memoria RAM que muestre de forma gráfica cómo se llenan los arreglos o *structs*; y tercero, un visualizador de la traducción automática entre pseudocódigo (PSeInt) y código real (C++/Java). Esto transformará un problema técnico real en un entorno de aprendizaje gamificado y altamente comprensible para los desarrolladores novatos.

### **1.11 Referencias bibliográficas**

[1] I. Author, "Editing SW-AyLP-APE-04--Clases-Objetos-y-M-todos/README.md," *GitHub*, May 2026. [Online]. Available: <https://github.com/isma1402uta-boop/SW-AyLP-APE-04--Clases-Objetos-y-M-todos/blob/main/README.md>. [Accessed: May 20, 2026].



## 1. ANALISIS DEL EJERCICIO PLANTEADO

### 1.1 Definición y Objetivo del Problema

Desarrollar un sistema informático interactivo, basado en el paradigma de Programación Orientada a Objetos (POO), que permita el registro continuo de estudiantes y sus calificaciones parciales. El sistema debe garantizar la integridad de los datos mediante validaciones estrictas (incluyendo la autenticidad de la cédula ecuatoriana), calcular el rendimiento académico de cada alumno, generar un reporte estadístico global y persistir la información físicamente en un archivo plano (CSV) para evitar la pérdida de datos.

### 1.2 Datos de Entrada (Inputs)

Por cada estudiante que se desea registrar, el sistema solicita los siguientes datos a través de la consola:

- **Cédula:** Cadena de caracteres numéricos (10 dígitos).
- **Nombres y Apellidos:** Cadena de caracteres alfabéticos (sin números ni símbolos).
- **Nota Parcial 1, Nota Parcial 2 y Nota Parcial 3:** Valores numéricos de punto flotante (float o double).
- **Cantidad de estudiantes a registrar:** Valor numérico entero (int).
- **Opción del menú:** Valor numérico entero (int).

### 1.3 Procesos y Cálculos Internos

El núcleo lógico del sistema ejecuta las siguientes operaciones tras recibir los datos de entrada:

**Validación de Identidad:** Se aplica el algoritmo matemático de *Coefficientes Módulo 10* sobre los 9 primeros dígitos de la cédula ingresada para calcular el dígito verificador y compararlo con el décimo dígito real.

**Cálculo del Promedio:** Se suman las tres notas parciales y se dividen para tres.

$$Promedio = \frac{Nota_1 + Nota_2 + Nota_3}{3}$$

**Determinación de Estado Académico:** Se evalúa el promedio calculado mediante una estructura condicional.

- Si  $Promedio \geq 7.0$ , el estado es "APROBADO".
- Si  $Promedio < 7.0$ , el estado es "REPROBADO".

**Persistencia Física:** Los datos procesados se concatenan en una cadena separada por comas y se envían a un flujo de escritura en modo anexión (*append*) hacia el archivo registro\_notas.csv.

**Contadores Estadísticos:** Durante la generación del reporte, el sistema recorre la colección en memoria contando iterativamente las frecuencias de los estados ("APROBADO" y



"REPROBADO").

### 1.4 Datos de Salida (Outputs)

El sistema devuelve los siguientes resultados al usuario:

**Alertas de Consola:** Mensajes de confirmación de guardado exitoso o de error descriptivo en caso de fallar alguna validación.

**Archivo Físico (registro\_notas.csv):** Un documento con los datos históricos de todos los alumnos registrados, compatible con hojas de cálculo.

**Reporte en Pantalla:** Una impresión tabular y detallada de cada estudiante registrado en la memoria actual.

**Resumen Estadístico:** \* Total de alumnos registrados.

- Total, de alumnos aprobados.
- Total, de alumnos reprobados.

### 1.5 Restricciones y Condiciones (Reglas de Negocio)

Para que el flujo del programa sea considerado válido, el código impone de forma automática las siguientes restricciones:

- **Rango de Notas:** Ninguna calificación parcial puede ser menor a 0.00 ni mayor a 10.00.
- **Blindaje de Tipado:** El programa no debe cerrarse abruptamente si se ingresan letras en campos de menús o de calificaciones numéricas.
- **Condición de Reporte:** El sistema bloquea la visualización del listado completo y las estadísticas si el tamaño de la lista de alumnos registrados en memoria es estrictamente menor a la constante definida (Mínimo global = 5 estudiantes).

## 2. DEFINICIÓN DE VARIABLES DENTRO DEL PROBLEMA

**TABLA I**

*Implementación de variables dentro del sistema Java - C++*

| VARIABLE<br>CONSTANTE | TIPO EN C++ | TIPO EN JAVA     | UBICACIÓN<br>ENTIDAD | DESCRIPCIÓN<br>PROPÓSITO<br>O LÓGICO                                                               |
|-----------------------|-------------|------------------|----------------------|----------------------------------------------------------------------------------------------------|
| MIN_ESTUDIANTES       | const int   | static final int | main<br>Main         | Constante que define la regla de negocio del sistema: el número mínimo de alumnos para desbloquear |



**UNIVERSIDAD TÉCNICA DE AMBATO**  
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL  
CARRERA DE SOFTWARE  
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



|                              |                                       |                                         |                         |                                                                                                                                                       |
|------------------------------|---------------------------------------|-----------------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LISTAESTUDIAN<br/>TES</b> | <b>vector&lt;Estudia<br/>ntes&gt;</b> | <b>ArrayList&lt;Estudi<br/>ante&gt;</b> | <b>main<br/>Main</b>    | / los reportes.<br>Colección<br>dinámica en<br>memoria<br>RAM que<br>almacena y<br>gestiona<br>todos los<br>objetos de los<br>alumnos<br>registrados. |
| <b>OPCION</b>                | <b>int</b>                            | <b>int</b>                              | <b>main<br/>Main</b>    | / Captura la<br>elección<br>numérica del<br>usuario para<br>controlar el<br>flujo del<br>switch en el<br>menú<br>principal.                           |
| <b>CANTIDAD</b>              | <b>int</b>                            | <b>int</b>                              | <b>main<br/>Main</b>    | / Determina el<br>límite del<br>bucle for al<br>registrar<br>alumnos por<br>lotes<br>(tandas).                                                        |
| <b>TOTALAPROBA<br/>DOS</b>   | <b>int</b>                            | <b>int</b>                              | <b>main<br/>Main</b>    | / Contador<br>estadístico<br>que se<br>incrementa si<br>el estudiante<br>cumple la<br>condición de<br>promedio >=<br>7.0.                             |
| <b>TOTALREPROB<br/>ADOS</b>  | <b>int</b>                            | <b>int</b>                              | <b>main<br/>Main</b>    | / Contador<br>estadístico<br>que se<br>incrementa si<br>el estudiante<br>no alcanza la<br>nota mínima<br>de<br>aprobación.                            |
| <b>CEDULA</b>                | <b>string</b>                         | <b>String</b>                           | <b>Estudiante<br/>s</b> | Identificador<br>único del<br>alumno. Se<br>procesa como<br>texto para<br>facilitar la<br>extracción y                                                |



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



|                            |               |               |                               |                                                                                                                         |
|----------------------------|---------------|---------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------|
|                            |               |               |                               | validación de sus dígitos.                                                                                              |
| <b>NOMBRE</b>              | <b>string</b> | <b>String</b> | <b>Estudiante</b><br><b>s</b> | Almacena los nombres del alumno tras pasar el filtro de validación alfabética.                                          |
| <b>APELLIDO</b>            | <b>string</b> | <b>String</b> | <b>Estudiante</b><br><b>s</b> | Almacena los apellidos del alumno tras pasar el filtro de validación alfabética.                                        |
| <b>NOTA1, NOTA2, NOTA3</b> | <b>float</b>  | <b>float</b>  | <b>Estudiante</b><br><b>s</b> | Calificaciones parciales del estudiante, fuertemente tipadas y restringidas al rango [0.00 - 10.00].                    |
| <b>PROMEDIO</b>            | <b>float</b>  | <b>float</b>  | <b>Estudiante</b><br><b>s</b> | Variable de solo lectura externa, calculada internamente por la clase como la media aritmética de las notas.            |
| <b>PROVINCIA</b>           | <b>int</b>    | <b>int</b>    | <b>Validador</b>              | Almacena los dos primeros caracteres de la cédula convertidos a entero para validar la región (01-24 o 30).             |
| <b>COEFICIENTES</b>        | <b>int[9]</b> | <b>int[]</b>  | <b>Validador</b>              | Arreglo estático de inicialización fija que contiene los multiplicadores [2, 1, 2, 1, 2, 1, 2, 1, 2] para el Módulo 10. |
| <b>SUMA</b>                | <b>int</b>    | <b>int</b>    | <b>Validador</b>              | Acumulador                                                                                                              |





|                                |                 |                   |                      |                                                                                                                                                      |
|--------------------------------|-----------------|-------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                |                 |                   |                      | temporal<br>utilizado para<br>procesar la<br>suma total<br>del algoritmo<br>del Registro<br>Civil.                                                   |
| <b>ARCHIVOBACK<br/>UP / FW</b> | <b>ofstream</b> | <b>FileWriter</b> | <b>main<br/>Main</b> | / Objeto<br>instanciado<br>para abrir el<br>canal de<br>comunicación<br>con el disco<br>duro y<br>escribir el<br>archivo CSV<br>en modo<br>"Append". |

*Nota: Las diferencias de tipado primitivo entre lenguajes son mínimas, pero el manejo de colecciones y objetos de texto presenta variaciones propias de la sintaxis nativa de cada entorno (vector vs ArrayList).*

### 3. ALGORITMO LÓGICO DENTRO DEL EJERCICIO PLANTEADO

#### Algoritmo Estructurado: Sistema de Calificaciones

##### Fase 1: Inicialización del Sistema

1. Definir la constante global requerida para habilitar los reportes (Mínimo = 5 estudiantes).
2. Inicializar la estructura de memoria dinámica (lista vacía) para almacenar los registros durante la ejecución.
3. Establecer el nombre del archivo físico en el disco duro para guardar los datos (registro\_notas.csv).

##### Fase 2: Menú Principal e Interacción

1. Iniciar un ciclo de repetición principal que se mantenga activo mientras la opción del usuario sea diferente a "3" (Salir).
2. **Mostrar en pantalla las opciones:** Ingresar Estudiantes, Mostrar Reportes, Salir.
3. Leer la opción ingresada por el usuario aplicando una limpieza del teclado para evitar que el programa falle si se ingresan letras por accidente.

##### Fase 3: Ejecución de la Opción 1 (Ingreso de Estudiantes)

1. Si la opción es 1, solicitar al usuario la cantidad de estudiantes que desea registrar en ese lote.



2. Iniciar un ciclo secundario que se repita exactamente la cantidad de veces solicitada.
3. Leer y validar la cédula: repetir la lectura de este campo hasta que el texto tenga 10 caracteres numéricos y el resultado del Algoritmo Módulo 10 sea verdadero
4. Leer y validar los nombres y apellidos: repetir la lectura hasta que el texto ingresado solo contenga caracteres alfabéticos, sin números ni símbolos.
5. Leer y validar las tres notas parciales: repetir la lectura de las calificaciones hasta que sean valores numéricos estrictamente dentro del rango de 0.00 a 10.00.
6. Calcular el promedio del estudiante sumando las tres notas y dividiendo el resultado para tres
7. Evaluar la condición académica: si el promedio es mayor o igual a 7.0, asignar el estado "APROBADO"; en caso contrario, asignar "REPROBADO".
8. Instanciar el objeto del estudiante con todos sus datos y guardarlo en la memoria dinámica del sistema.
9. Abrir el archivo físico "registro\_notas.csv", agregar la nueva línea de información separada por comas y cerrar el archivo para asegurar el guardado.

#### **Fase 4: Ejecución de la Opción 2 (Reportes y Estadísticas)**

1. Si la opción es 2, contar la cantidad total de estudiantes actualmente guardados en la memoria.
2. Si la cantidad almacenada es menor a 5, mostrar un mensaje de error por pantalla y regresar inmediatamente al menú principal.
3. Si la cantidad almacenada cumple con el mínimo exigido, inicializar los contadores estadísticos de "total de aprobados" y "total de reprobados" en cero.
4. Recorrer la lista de estudiantes en memoria de inicio a fin
5. Imprimir en pantalla los datos personales y académicos de cada estudiante evaluado en el recorrido.
6. Sumar 1 al contador respectivo dependiendo de si el estado del estudiante actual es "APROBADO" o "REPROBADO".
7. Imprimir en pantalla el resumen estadístico final detallando el total de alumnos registrados, el total de aprobados y el total de reprobados.

#### **Fase 5: Validación y Finalización del Sistema**

1. Si la opción del usuario es 3, mostrar un mensaje de despedida y romper el ciclo de repetición principal.
2. Si la opción ingresada no es 1, 2 ni 3, mostrar un mensaje de error indicando que la opción es inválida y volver a mostrar el menú.



- Al romper el ciclo principal, finalizar la ejecución del programa y liberar los recursos de memoria del sistema.

#### 4. ALGORITMO EN EL PROGRAMA PSEINT

Algoritmo SistemaAcademico

// Definición de variables primitivas

Definir opcion, cantidad, i, total\_estudiantes, aprobados, reprobados,  
MIN\_ESTUDIANTES Como Entero;

Definir nota1, nota2, nota3 Como Real;

// Inicialización de reglas de negocio

MIN\_ESTUDIANTES <- 5;

total\_estudiantes <- 0;

// Definición de Arreglos Paralelos para simular la memoria dinámica

Dimension cedulas[100];

Dimension nombres[100];

Dimension promedios[100];

Dimension estados[100];

Definir cedulas, nombres, estados Como Caracter;

Definir promedios Como Real;

// Bucle Principal del Sistema

Repetir

Escribir "===== MENÚ PRINCIPAL =====";

Escribir "1. Registrar Estudiantes";

Escribir "2. Mostrar Reporte y Estadísticas";

Escribir "3. Salir";

Escribir "Seleccione una opción:";

Leer opcion;

Segun opcion Hacer

1:

Escribir "¿Cuántos estudiantes desea registrar en esta tanda?";

Leer cantidad;

Para i <- 1 Hasta cantidad Con Paso 1 Hacer

Escribir "--- Datos del Estudiante ---";

Escribir "Ingrese Cédula:";

Leer cedulas[total\_estudiantes + 1];

Escribir "Ingrese Nombres y Apellidos:";

Leer nombres[total\_estudiantes + 1];

Escribir "Ingrese Nota Parcial 1:";

Leer nota1;

Escribir "Ingrese Nota Parcial 2:";

Leer nota2;

Escribir "Ingrese Nota Parcial 3:";

Leer nota3;



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



```
// Cálculo Lógico
promedios[total_estudiantes + 1] <- (nota1 + nota2 + nota3) / 3;

// Evaluación de Estado
Si promedios[total_estudiantes + 1] >= 7.0 Entonces
    estados[total_estudiantes + 1] <- "APROBADO";
SiNo
    estados[total_estudiantes + 1] <- "REPROBADO";
FinSi

total_estudiantes <- total_estudiantes + 1;
Escribir "=> Estudiante registrado y guardado exitosamente.";
Escribir "";
FinPara

2:
// Validación de Regla de Negocio
Si total_estudiantes < MIN_ESTUDIANTES Entonces
    Escribir "ERROR: Faltan estudiantes. Mínimo requerido: ",
MIN_ESTUDIANTES;
    Escribir "Actualmente registrados: ", total_estudiantes;
    Escribir "";
SiNo
    aprobados <- 0;
    reprobados <- 0;

    Escribir "===== REPORTE ACADÉMICO =====";
    Para i <- 1 Hasta total_estudiantes Con Paso 1 Hacer
        Escribir "ID: ", cedulas[i], " | Nombre: ", nombres[i], " | Promedio: ",
promedios[i], " | Estado: ", estados[i];

        // Sumatoria Estadística
        Si estados[i] = "APROBADO" Entonces
            aprobados <- aprobados + 1;
        SiNo
            reprobados <- reprobados + 1;
        FinSi
    FinPara

    Escribir "===== ESTADÍSTICAS =====";
    Escribir "Total Registrados: ", total_estudiantes;
    Escribir "Total Aprobados: ", aprobados;
    Escribir "Total Reprobados: ", reprobados;
    Escribir "";
FinSi

3:
Escribir "Guardando datos... Saliendo del sistema. ¡Hasta pronto!";

De Otro Modo:
    Escribir "ERROR: Opción inválida. Ingrese un número del 1 al 3.";
    Escribir "";
FinSegun
```



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**

---



Hasta Que opcion = 3;

FinAlgoritmo

*Fig. 1. Diagrama de flujo detallado de la lógica de programación del sistema, generado mediante la herramienta PSeInt.*  
*Fuente: Elaboración propia.*



## 6. CODIFICACIÓN EN EL LENGUAJE DE PROGRAMACIÓN C++

- **Clase Estudiantes.cpp**

```
#include "Estudiantes.h"
#include <iostream>
#include <iomanip>

using namespace std;

// Constructor por defecto
Estudiantes::Estudiantes() {
    cedula = "";
    nombre = "";
    apellido = "";
    nota1 = 0.0f;
    nota2 = 0.0f;
    nota3 = 0.0f;
    promedio = 0.0f;
}

// Constructor parametrizado
Estudiantes::Estudiantes(string _cedula, string _nombre, string _apellido, float _n1, float
_n2, float _n3) {
    cedula = _cedula;
    nombre = _nombre;
    apellido = _apellido;
    nota1 = _n1;
    nota2 = _n2;
    nota3 = _n3;
    calcularPromedio();
}

// Métodos Get y Set
void Estudiantes::setCedula(string _cedula) { cedula = _cedula; }
string Estudiantes::getCedula() const { return cedula; }

void Estudiantes::setNombre(string _nombre) { nombre = _nombre; }
string Estudiantes::getNombre() const { return nombre; }

void Estudiantes::setApellido(string _apellido) { apellido = _apellido; }
string Estudiantes::getApellido() const { return apellido; }

void Estudiantes::setNota1(float _n1) { nota1 = _n1; calcularPromedio(); }
float Estudiantes::getNota1() const { return nota1; }

void Estudiantes::setNota2(float _n2) { nota2 = _n2; calcularPromedio(); }
float Estudiantes::getNota2() const { return nota2; }

void Estudiantes::setNota3(float _n3) { nota3 = _n3; calcularPromedio(); }
float Estudiantes::getNota3() const { return nota3; }

float Estudiantes::getPromedio() const { return promedio; }
```



```
void Estudiantes::calcularPromedio() {
    promedio = (nota1 + nota2 + nota3) / 3.0f;
}

bool Estudiantes::determinarAprobacion() const {
    return (promedio >= 7.0f);
}

void Estudiantes::mostrarInformacion() const {
    cout << "-----\n";
    cout << "Cedula: " << cedula << "\n";
    cout << "Alumno: " << apellido << " " << nombre << "\n";
    cout << fixed << setprecision(2);
    cout << "Nota 1: " << nota1 << " | Nota 2: " << nota2 << " | Nota 3: " << nota3 <<
    "\n";
    cout << "Promedio: " << promedio << "\n";

    if (determinarAprobacion()) {
        cout << "Estado: [APROBADO]\n";
    } else {
        cout << "Estado: [REPROBADO]\n";
    }
    cout << "-----\n";
}
```

- **Cabecera Estudiantes.h**

```
#ifndef ESTUDIANTES_H
#define ESTUDIANTES_H

#include <string>

class Estudiantes {
private:
    // Atributos privados protegidos (Encapsulamiento)
    std::string cedula;
    std::string nombre;
    std::string apellido;
    float nota1;
    float nota2;
    float nota3;
    float promedio;

public:
    // Constructores
    Estudiantes(); // Constructor por defecto
    Estudiantes(std::string _cedula, std::string _nombre, std::string _apellido, float _n1, float
    _n2, float _n3);

    // Métodos Get y Set
    void setCedula(std::string _cedula);
    std::string getCedula() const;
```





```
void setNombre(std::string _nombre);
std::string getNombre() const;

void setApellido(std::string _apellido);
std::string getApellido() const;

void setNota1(float _n1);
float getNota1() const;

void setNota2(float _n2);
float getNota2() const;

void setNota3(float _n3);
float getNota3() const;

// Solo se permite obtener el promedio, la modificación es interna
float getPromedio() const;

// Métodos de Lógica del Negocio
void calcularPromedio();
bool determinarAprobacion() const;
void mostrarInformacion() const;
};

#endif // ESTUDIANTES_H
```

- **Clase validadorEstudiante.cpp**

```
#include "validadorEstudiante.h"
#include <iostream>
#include <limits>
#include <cctype> // Necesaria para operaciones nativas de caracteres (isalpha, isspace)

using namespace std;

// Constructor
ValidadorEstudiante::ValidadorEstudiante() {}

// MÉTODO PRIVADO: Algoritmo matemático del Registro Civil de Ecuador
bool ValidadorEstudiante::esCedulaRealEcuatoriana(const string& cedula) {

    // 1. Validar código de provincia (Primeros dos dígitos)
    int provincia = (cedula[0] - '0') * 10 + (cedula[1] - '0');
    if ((provincia < 1 || provincia > 24) && provincia != 30) {
        return false;
    }

    // 2. Validar el tercer dígito (Debe ser menor a 6 para personas naturales)
    int tercerDigito = cedula[2] - '0';
    if (tercerDigito >= 6) {
        return false;
    }
}
```



```
// 3. Algoritmo de Coeficientes de Módulo 10
int coeficientes[9] = {2, 1, 2, 1, 2, 1, 2, 1, 2};
int suma = 0;

for (int i = 0; i < 9; i++) {
    int valor = (cedula[i] - '0') * coeficientes[i];
    if (valor > 9) {
        valor -= 9; // Si el producto es de dos dígitos, se restan 9
    }
    suma += valor;
}

// Calcular dígito verificador esperado
int digitoVerificadorCalculado = (suma % 10 == 0) ? 0 : 10 - (suma % 10);
int digitoVerificadorReal = cedula[9] - '0';

// Compara el cálculo matemático con el último dígito real
return (digitoVerificadorCalculado == digitoVerificadorReal);
}

// MÉTODO PÚBLICO: Captura y blindaje de Cédula en consola
string ValidadorEstudiante::validarCedula(const string& mensaje) {
    string cedula;
    while (true) {
        cout << mensaje;
        cin >> cedula;

        // Limpieza absoluta del búfer para que conviva perfectamente con getlines posteriores
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        // FILTRO 1: Longitud exacta
        if (cedula.length() != 10) {
            cout << "-> [ERROR] La cedula debe tener exactamente 10 digitos.\n";
        }
        // FILTRO 2: Solo caracteres del '0' al '9'
        else if (cedula.find_first_not_of("0123456789") != string::npos) {
            cout << "-> [ERROR] La cedula solo debe contener caracteres numericos.\n";
        }
        // FILTRO 3: Validación lógica real en el territorio nacional
        else if (!esCedulaRealEcuatoriana(cedula)) {
            cout << "-> [ERROR] El numero de cedula no existe o es invalido en Ecuador.\n";
        }
        else {
            return cedula; // Cédula totalmente legítima y segura
        }
    }
}

// MÉTODO PÚBLICO: Captura de Nombres y Apellidos en español
string ValidadorEstudiante::validarTexto(const string& mensaje) {
    string texto;
    while (true) {
```



```
cout << mensaje;
getline(cin, texto); // Permite capturar apellidos o nombres compuestos con espacios

// FILTRO 1: Validar que no se envíe en blanco o lleno de puros espacios virtuales
if (texto.empty() || texto.find_first_not_of(' ') == string::npos) {
    cout << "-> [ERROR] El campo no puede quedar vacio. Intente de nuevo.\n";
    continue;
}

bool caracteresValidos = true;

// FILTRO 2: Inspección analítica de cada letra
for (size_t i = 0; i < texto.length(); i++) {
    char c = texto[i];

    // OPTIMIZADO: Si es letra estándar, espacio o un carácter extendido (tildes/ñ
    // donde c < 0)
    if (isalpha(c) || isspace(c) || c < 0) {
        continue;
    }

    // Si encuentra un número o símbolo no autorizado, levanta la bandera de error
    caracteresValidos = false;
    break;
}

if (!caracteresValidos) {
    cout << "-> [ERROR] El campo solo debe contener letras y espacios (sin numeros ni
    simbolos).\n";
} else {
    return texto; // Retorna el string sanitizado y correcto
}
}
```

- **Cabecera validadorEstudiante.h**

```
#ifndef VALIDADOR_ESTUDIANTE_H
#define VALIDADOR_ESTUDIANTE_H

#include <string>

// CLASE: ValidadorEstudiante
// Objetivo: Blindar la identidad del alumno (Cédula, Nombres, Apellidos)
class ValidadorEstudiante {
private:
    // Método auxiliar matemático interno para verificar la cédula de Ecuador
    bool esCedulaRealEcuatoriana(const std::string& cedula);

public:
    // Constructor por defecto
    ValidadorEstudiante();
};
```



```
// Captura y valida estructuralmente la cédula en la consola
std::string validarCedula(const std::string& mensaje);

// Captura y valida nombres o apellidos (admite espacios, tildes y Ñ)
std::string validarTexto(const std::string& mensaje);
};

#endif // VALIDADOR_ESTUDIANTE_H
```

- **Clase validarNotas.cpp**

```
#include "validarNotas.h"
#include <iostream>
#include <limits>

using namespace std;

// CRUCIAL: Definir explícitamente el constructor de la clase
LectorDatos::LectorDatos() {}

int LectorDatos::validarEntero(const string &mensaje)
{
    int valor;
    while (true)
    {
        cout << mensaje;
        if (cin >> valor)
        {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return valor;
        }
        cout << "-> [ERROR] Ingreso invalido. Debe ser un numero entero.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

float LectorDatos::validarNota(const string &mensaje)
{
    float nota;
    while (true)
    {
        cout << mensaje;
        if (cin >> nota)
        {
            if (nota >= 0.0f && nota <= 10.0f)
            {
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                return nota;
            }
            cout << "-> [ERROR] La nota debe estar estrictamente en el rango de 0.00 a 10.00.\n";
        }
    }
}
```



```
else
{
    cout << "-> [ERROR] Ingreso invalido. Debe ser un numero decimal o entero.\n";
    cin.clear();
}
cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
}
```

- **Cabecera validarNotas.h**

```
#ifndef VALIDARNOTAS_H
#define VALIDARNOTAS_H

#include <string>

// CLASE: LectorDatos
// Objetivo: Declarar los métodos que blindarán la entrada de datos

class LectorDatos {
public:
    // Constructor por defecto
    LectorDatos();

    // Valida y lee un número entero (ideal para el menú principal)
    int validarEntero(const std::string& mensaje);

    // Valida y lee una nota flotante entre 0 y 10 (Requerimiento APE 04)
    float validarNota(const std::string& mensaje);
};

#endif // VALIDARNOTAS_H
```

- **Clase main.cpp**

```
#include <iostream>
#include <vector>
#include "Estudiantes.h" // Estructura y lógica del alumno
#include "validadorEstudiante.h" // Validación de cédula real, nombres y apellidos
#include "validarNotas.h" // Clase encargada de validar números y rangos de notas
#include <fstream> // Librería para manejo de archivos

using namespace std;

// Constante global para asegurar el requerimiento mínimo de la guía práctica
const int MIN_ESTUDIANTES = 5;

int main() {
    // Instanciamos los objetos encargados de la seguridad y captura de datos
    LectorDatos lector; // Maneja entradas numéricas de notas y menús
    ValidadorEstudiante validadorEst; // Maneja la rigurosidad de los datos personales

    // Vector dinámico para almacenar la lista de objetos de tipo Estudiantes
```



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



```
vector<Estudiantes> listaEstudiantes;

int opcion;
do {

                                                                    cout <<
"\n===== \n";
    cout << " SISTEMA ACADEMICO DE CALIFICACIONES (APE 04) \n";
    cout << "===== \n";
    cout << "1. Registrar Estudiantes (Minimo global: " << MIN_ESTUDIANTES <<
")\n";
    cout << "2. Mostrar Listado Completo y Estadisticas de Notas\n";
    cout << "3. Salir del Sistema\n";
    cout << "-----\n";

// REQUERIMIENTO 7: Validar la entrada del menú principal
opcion = lector.validarEntero("Seleccione una opcion: ");
cout << endl;

switch (opcion) {
    case 1: {
        cout << "=== REGISTRO DE NUEVOS ESTUDIANTES ===\n";
        cout << "Estudiantes registrados actualmente: " << listaEstudiantes.size() << "\n";
        int cantidad = lector.validarEntero("Cual es la cantidad de estudiantes a registrar
ahora?: ");

// REQUERIMIENTO 1 OPTIMIZADO: Valida que el total acumulado alcance el
mínimo exigido
        while ((listaEstudiantes.size() + cantidad) < MIN_ESTUDIANTES) {
            cout << "-> [ERROR] El sistema requiere un total minimo acumulado de " <<
MIN_ESTUDIANTES << " estudiantes.\n";
            cout << " Faltan al menos " << (MIN_ESTUDIANTES -
listaEstudiantes.size()) << " por registrar.\n";
            cantidad = lector.validarEntero("Por favor, ingrese una cantidad valida: ");
        }

// REQUERIMIENTO 1 OPTIMIZADO: Reserva de memoria anticipada para
evitar realojamientos costosos
        listaEstudiantes.reserve(listaEstudiantes.size() + cantidad);

// Ciclo estructurado para capturar la información de cada alumno
        for (int i = 0; i < cantidad; i++) {
            cout << "\n--- REGISTRANDO ESTUDIANTE #" << (i + 1) << " DE " <<
cantidad << " ---\n";

// Captura con validación avanzada de identidad (Cédula real de Ecuador)
            string cedula = validadorEst.validarCedula("Ingrese el numero de cedula: ");
            string nombre = validadorEst.validarTexto("Ingrese los nombres: ");
            string apellido = validadorEst.validarTexto("Ingrese los apellidos: ");

// REQUERIMIENTO 2 Y 7: Ingresar las 3 notas validando el rango (0.00 -
10.00)
            float n1 = lector.validarNota("Ingrese la Nota del Parcial 1 (0.00 - 10.00): ");
            float n2 = lector.validarNota("Ingrese la Nota del Parcial 2 (0.00 - 10.00): ");
```



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



```
float n3 = lector.validarNota("Ingrese la Nota del Parcial 3 (0.00 - 10.00): ");

// REQUERIMIENTO 3: El constructor parametrizado calcula internamente el
promedio
Estudiantes nuevoAlumno(cedula, nombre, apellido, n1, n2, n3);

// Almacenamiento eficiente de O(1) gracias a la reserva previa de memoria
listaEstudiantes.push_back(nuevoAlumno);
cout << "-> [EXITO] Estudiante almacenado correctamente.\n";

// Abrimos (o creamos) el archivo en modo 'app' (append = añadir al final)
ofstream archivoBackup("registro_notas.csv", ios::app);

if (archivoBackup.is_open()) {
    // Guardamos los datos separados por comas para que sea compatible con
Excel
    archivoBackup << cedula << ","
        << apellido << " " << nombre << ","
        << n1 << "," << n2 << "," << n3 << ","
        << nuevoAlumno.getPromedio() << ","
        << (nuevoAlumno.determinarAprobacion() ? "APROBADO" :
"REPROBADO") << "\n";

    archivoBackup.close(); // Siempre es vital cerrar el archivo para liberar la
memoria
    cout << "-> [EXITO] Respaldo guardado físicamente en
'registro_notas.csv'.\n";
} else {
    cout << "-> [ERROR CRITICO] No se pudo acceder al disco para guardar el
archivo.\n";
}
}
break;
}
case 2: {
    // Validación de seguridad por si intentan ver reportes sin datos previos
    if (listaEstudiantes.empty()) {
        cout << "-> [AVISO] El sistema no registra alumnos guardados. Primero use la
opcion [1].\n";
    } else {
        cout <<
"===== \n";
        cout << "    REQUERIMIENTO 4: LISTADO DE ESTUDIANTES    \n";
        cout <<
"===== \n";

        int totalAprobados = 0;
        int totalReprobados = 0;

        // Bucle para recorrer el vector e imprimir las fichas académicas
        for (size_t i = 0; i < listaEstudiantes.size(); i++) {
            listaEstudiantes[i].mostrarInformacion();
        }
    }
}
```



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



```
// Clasificación de estados para las métricas finales
if (listaEstudiantes[i].determinarAprobacion()) {
    totalAprobados++;
} else {
    totalReprobados++;
}
}

// REQUERIMIENTO 5 Y 6: Presentación resumida de contadores estadísticos
cout <<
"\n===== \n";
cout << "          RESUMEN ESTADISTICO          \n";
cout <<
"===== \n";
cout << " Total de estudiantes en el listado: " << listaEstudiantes.size() << "\n";
cout << " REQUERIMIENTO 5: Alumnos Aprobados: " << totalAprobados <<
"\n";
cout << " REQUERIMIENTO 6: Alumnos Reprobados: " << totalReprobados
<< "\n";
cout <<
"===== \n";
}
break;
}
case 3:
    cout << "-> Cerrando sesion. El sistema de calificaciones se ha cerrado
correctamente.\n";
    break;

default:
    cout << "-> [ERROR] Opcion fuera de rango. Por favor elija [1, 2 o 3].\n";
    break;
}

} while (opcion != 3);

return 0;
}
```





## 7. PRUEBAS DE SOLUCIÓN DEL SISTEMA DESARROLLADO C++

- Menú del sistema propuesto en C++

```
=====
SISTEMA ACADEMICO DE CALIFICACIONES (APE 04)
=====
1. Registrar Estudiantes (Minimo global: 5)
2. Mostrar Listado Completo y Estadísticas de Notas
3. Salir del Sistema
-----
Seleccione una opcion: 1
```

Fig. 2. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado [1].

Fuente: Elaboración propia.

- Registro de 4 estudiantes dentro del sistema propuesto en C++

```
--- REGISTRANDO ESTUDIANTE #1 DE 5 ---
Ingrese el numero de cedula: 0504341272
Ingrese los nombres: Isaac Adrian
Ingrese los apellidos: Sanchez Lema
Ingrese la Nota del Parcial 1 (0.00 - 10.00): 9
Ingrese la Nota del Parcial 2 (0.00 - 10.00): 8
Ingrese la Nota del Parcial 3 (0.00 - 10.00): 9
-> [EXITO] Estudiante almacenado correctamente.
-> [EXITO] Respaldo guardado fisicamente en 'registro_notas.csv'.

--- REGISTRANDO ESTUDIANTE #2 DE 5 ---
Ingrese el numero de cedula: 0504341272
Ingrese los nombres: Ismael Andres
Ingrese los apellidos: Nuñez Lema
Ingrese la Nota del Parcial 1 (0.00 - 10.00): 4
Ingrese la Nota del Parcial 2 (0.00 - 10.00): 5
Ingrese la Nota del Parcial 3 (0.00 - 10.00): 6
-> [EXITO] Estudiante almacenado correctamente.
-> [EXITO] Respaldo guardado fisicamente en 'registro_notas.csv'.

--- REGISTRANDO ESTUDIANTE #3 DE 5 ---
Ingrese el numero de cedula: 0504341272
Ingrese los nombres: Melany Sarahi
Ingrese los apellidos: Garcia Escobar
Ingrese la Nota del Parcial 1 (0.00 - 10.00): 7
Ingrese la Nota del Parcial 2 (0.00 - 10.00): 8
Ingrese la Nota del Parcial 3 (0.00 - 10.00): 9
-> [EXITO] Estudiante almacenado correctamente.
-> [EXITO] Respaldo guardado fisicamente en 'registro_notas.csv'.

--- REGISTRANDO ESTUDIANTE #4 DE 5 ---
Ingrese el numero de cedula: 0504341272
Ingrese los nombres: Mishell Fernanda
Ingrese los apellidos: Vilca Nuñez
Ingrese la Nota del Parcial 1 (0.00 - 10.00): 9
Ingrese la Nota del Parcial 2 (0.00 - 10.00): 86
-> [ERROR] La nota debe estar estrictamente en el rango de 0.00 a 10.00.
Ingrese la Nota del Parcial 2 (0.00 - 10.00): 7
Ingrese la Nota del Parcial 3 (0.00 - 10.00): 6.78
-> [EXITO] Estudiante almacenado correctamente.
-> [EXITO] Respaldo guardado fisicamente en 'registro_notas.csv'.
```

Fig. 3. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado mediante validaciones de entradas

Fuente: Elaboración propia.



- Resumen de la cantidad de estudiantes registrados dentro del sistema

```
=====
                        REQUERIMIENTO 4: LISTADO DE ESTUDIANTES
=====

-----
Cedula: 0504341272
Alumno: Sanchez Lema Isaac Adrian
Nota 1: 9.00 | Nota 2: 8.00 | Nota 3: 9.00
Promedio: 8.67
Estado: [APROBADO]
-----

-----
Cedula: 0504341272
Alumno: Nuñez Lema Ismael Andres
Nota 1: 4.00 | Nota 2: 5.00 | Nota 3: 6.00
Promedio: 5.00
Estado: [REPROBADO]
-----

-----
Cedula: 0504341272
Alumno: Garcia Esocobar Melany Sarahi
Nota 1: 7.00 | Nota 2: 8.00 | Nota 3: 9.00
Promedio: 8.00
Estado: [APROBADO]
-----

-----
Cedula: 0504341272
Alumno: Vilca Nuñez Mishell Fernanda
Nota 1: 9.00 | Nota 2: 7.00 | Nota 3: 6.78
Promedio: 7.59
Estado: [APROBADO]
-----

-----
Cedula: 0504341272
Alumno: Preciado Cuenca Artes Luis
Nota 1: 8.00 | Nota 2: 9.00 | Nota 3: 0.00
Promedio: 5.67
Estado: [REPROBADO]
-----

=====
                        RESUMEN ESTADISTICO
=====

Total de estudiantes en el listado: 5
REQUERIMIENTO 5: Alumnos Aprobados: 3
REQUERIMIENTO 6: Alumnos Reprobados: 2
=====
```

*Fig. 4. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado mediante validación de datos de entrada múltiple.  
Fuente: Elaboración propia.*



## 8. CODIFICACIÓN EN EL LENGUAJE DE PROGRAMACIÓN JAVA

- **Clase Estudiante.java**

```
public class Estudiante {
    // Atributos privados
    private String cedula;
    private String nombre;
    private String apellido;
    private float nota1;
    private float nota2;
    private float nota3;
    private float promedio;

    // Constructor por defecto
    public Estudiante() {
        this.cedula = "";
        this.nombre = "";
        this.apellido = "";
        this.nota1 = 0.0f;
        this.nota2 = 0.0f;
        this.nota3 = 0.0f;
        this.promedio = 0.0f;
    }

    // Constructor parametrizado
    public Estudiante(String cedula, String nombre, String apellido, float n1, float n2, float
n3) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.apellido = apellido;
        this.nota1 = n1;
        this.nota2 = n2;
        this.nota3 = n3;
        calcularPromedio();
    }

    // Métodos lógicos matemáticos
    private void calcularPromedio() {
        this.promedio = (this.nota1 + this.nota2 + this.nota3) / 3.0f;
    }

    public boolean determinarAprobacion() {
        return this.promedio >= 7.0f;
    }

    // Impresión formateada (Reemplaza a <iomanip>)
    public void mostrarInformacion() {
        System.out.println("-----");
        System.out.println("Cedula: " + cedula);
        System.out.println("Alumno: " + apellido + " " + nombre);
        System.out.printf("Nota 1: %.2f | Nota 2: %.2f | Nota 3: %.2f\n", nota1, nota2,
nota3);
    }
}
```



```
System.out.printf("Promedio: %.2f\n", promedio);
System.out.println("Estado: " + (determinarAprobacion() ? "[APROBADO]" :
"[REPROBADO]"));
System.out.println("-----");
}

// Getters y Setters
public String getCedula() { return cedula; }
public void setCedula(String cedula) { this.cedula = cedula; }

public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }

public String getApellido() { return apellido; }
public void setApellido(String apellido) { this.apellido = apellido; }

public float getNota1() { return nota1; }
public void setNota1(float nota1) { this.nota1 = nota1; calcularPromedio(); }

public float getNota2() { return nota2; }
public void setNota2(float nota2) { this.nota2 = nota2; calcularPromedio(); }

public float getNota3() { return nota3; }
public void setNota3(float nota3) { this.nota3 = nota3; calcularPromedio(); }

public float getPromedio() { return promedio; }
}
```

- **Clase LectorDatos.java**

```
import java.util.Scanner;

public class LectorDatos {
    private Scanner teclado;

    public LectorDatos() {
        this.teclado = new Scanner(System.in);
    }

    public int validarEntero(String mensaje) {
        int valor;
        while (true) {
            System.out.print(mensaje);
            try {
                valor = Integer.parseInt(teclado.nextLine().trim());
                return valor;
            } catch (NumberFormatException e) {
                System.out.println("-> [ERROR] Ingreso invalido. Debe ser un numero entero.");
            }
        }
    }

    public float validarNota(String mensaje) {
```



```
float nota;
while (true) {
    System.out.print(mensaje);
    try {
        nota = Float.parseFloat(teclado.nextLine().trim());
        if (nota >= 0.0f && nota <= 10.0f) {
            return nota;
        }
        System.out.println("-> [ERROR] La nota debe estar estrictamente en el rango de
0.00 a 10.00.");
    } catch (NumberFormatException e) {
        System.out.println("-> [ERROR] Ingreso invalido. Debe ser un numero decimal o
entero.");
    }
}
}
```

- **Clase ValidadorEstudiante.java**

```
import java.util.Scanner;
```

```
public class ValidadorEstudiante {
    // Atributo privado único para el manejo de la consola (Evita fugas de memoria)
    private final Scanner sc;

    // Constructor que inicializa el lector de forma segura
    public ValidadorEstudiante() {
        this.sc = new Scanner(System.in);
    }
}
```

```
// Algoritmo de validación del Registro Civil (Módulo 10)
private boolean esCedulaRealEcuatoriana(String cedula) {
    // 1. Código de provincia (01 a 24, o 30 para ecuatorianos en el exterior)
    int provincia = Integer.parseInt(cedula.substring(0, 2));
    if ((provincia < 1 || provincia > 24) && provincia != 30) {
        return false;
    }
}
```

```
// 2. Tercer dígito menor a 6 (Cédulas de personas naturales)
int tercerDigito = Character.getNumericValue(cedula.charAt(2));
if (tercerDigito >= 6) {
    return false;
}
```

```
// 3. Algoritmo de coeficientes módulo 10
int[] coeficientes = {2, 1, 2, 1, 2, 1, 2, 1, 2};
int suma = 0;
```

```
for (int i = 0; i < 9; i++) {
    int valor = Character.getNumericValue(cedula.charAt(i)) * coeficientes[i];
    if (valor > 9) {
        valor -= 9;
    }
}
```



```
}
    suma += valor;
}

int digitoVerificadorCalculado = (suma % 10 == 0) ? 0 : 10 - (suma % 10);
int digitoVerificadorReal = Character.getNumericValue(cedula.charAt(9));

return digitoVerificadorCalculado == digitoVerificadorReal;
}

// CORRECCIÓN: Se removi6 el parámetro LectorDatos innecesario y se usa el 'sc' global
public String validarCedula(String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String cedula = sc.nextLine().trim();

        if (cedula.length() != 10) {
            System.out.println("-> [ERROR] La cedula debe tener exactamente 10 digitos.");
        } else if (!cedula.matches("[0-9]+")) {
            System.out.println("-> [ERROR] La cedula solo debe contener caracteres
numericos.");
        } else if (!esCedulaRealEcuatoriana(cedula)) {
            System.out.println("-> [ERROR] El numero de cedula no existe o es invalido en
Ecuador.");
        } else {
            return cedula;
        }
    }
}

public String validarTexto(String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String texto = sc.nextLine();

        if (texto.trim().isEmpty()) {
            System.out.println("-> [ERROR] El campo no puede quedar vacio. Intente de
nuevo.");
            continue;
        }

        boolean caracteresValidos = true;
        for (int i = 0; i < texto.length(); i++) {
            char c = texto.charAt(i);

            // Reconoce de manera nativa tildes, diéresis y la Ñ
            if (Character.isLetter(c) || Character.isWhitespace(c)) {
                continue;
            }
            caracteresValidos = false;
            break;
        }
    }
}
```



```
if (!caracteresValidos) {
    System.out.println("-> [ERROR] El campo solo debe contener letras y espacios
(sin numeros ni simbolos).");
} else {
    return texto.trim(); // Retorna el texto limpio de espacios innecesarios a los lados
}
}
}
```

- **Clase Main.java**

```
import java.util.ArrayList;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.io.IOException;

public class Main {
    private static final int MIN_ESTUDIANTES = 5;

    public static void main(String[] args) {
        LectorDatos lector = new LectorDatos();
        ValidadorEstudiante validadorEst = new ValidadorEstudiante();

        // ArrayList dinámico para el almacenamiento en memoria RAM
        ArrayList<Estudiante> listaEstudiantes = new ArrayList<>();

        int opcion;
        do {
            System.out.println("\n=====
=====");
            System.out.println(" SISTEMA ACADEMICO DE CALIFICACIONES (JAVA -
APE) ");
            System.out.println("=====
=====");
            System.out.println("1. Registrar Estudiantes (Minimo global: " +
MIN_ESTUDIANTES + ")");
            System.out.println("2. Mostrar Listado Completo y Estadisticas de Notas");
            System.out.println("3. Salir del Sistema");
            System.out.println("-----");

            opcion = lector.validarEntero("Seleccione una opcion: ");
            System.out.println();

            switch (opcion) {
                case 1:
                    System.out.println("== REGISTRO DE NUEVOS ESTUDIANTES ==");
                    System.out.println("Estudiantes registrados actualmente: " +
listaEstudiantes.size());
                    int cantidad = lector.validarEntero("Cual es la cantidad de estudiantes a registrar
ahora?: ");

                    // Alta eficiencia: Reserva espacio contiguo inicial en la colección
```



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



```
listaEstudiantes.ensureCapacity(listaEstudiantes.size() + cantidad);

for (int i = 0; i < cantidad; i++) {
    System.out.println("\n--- REGISTRANDO ESTUDIANTE #" + (i + 1) + "
DE " + cantidad + " ---");

    // ValidadorEstudiante
    String cedula = validadorEst.validarCedula("Ingrese el numero de cedula: ");
    String nombre = validadorEst.validarTexto("Ingrese los nombres: ");
    String apellido = validadorEst.validarTexto("Ingrese los apellidos: ");

    float n1 = lector.validarNota("Ingrese la Nota del Parcial 1 (0.00 - 10.00): ");
    float n2 = lector.validarNota("Ingrese la Nota del Parcial 2 (0.00 - 10.00): ");
    float n3 = lector.validarNota("Ingrese la Nota del Parcial 3 (0.00 - 10.00): ");

    // Instanciación y cálculo interno automático del promedio
    Estudiante nuevoAlumno = new Estudiante(cedula, nombre, apellido, n1, n2,
n3);
    listaEstudiantes.add(nuevoAlumno);
    System.out.println("-> [EXITO] Estudiante almacenado correctamente en
memoria.");

    // Módulo de persistencia física de datos (Modo Append activo)
    try (FileWriter fw = new FileWriter("registro_notas.csv", true);
        PrintWriter escritor = new PrintWriter(fw)) {

        // Formato CSV estructurado por comas para compatibilidad inmediata con
MS Excel
        escritor.println(cedula + ","
            + apellido + " " + nombre + ","
            + n1 + "," + n2 + "," + n3 + ","
            + nuevoAlumno.getPromedio() + ","
            + (nuevoAlumno.determinarAprobacion() ? "APROBADO" :
"REPROBADO"));

        System.out.println("-> [EXITO] Respaldo guardado físicamente en
'registro_notas.csv'.");

    } catch (IOException e) {
        System.out
.println("-> [ERROR CRITICO] No se pudo acceder al disco duro: "
+ e.getMessage());
    }
}
break;

case 2:
    // Escudo de control de tamaño mínimo flexible solicitado por la guía
    if (listaEstudiantes.size() < MIN_ESTUDIANTES) {
        System.out.println(
            "-> [ERROR] No se puede mostrar el reporte. El sistema exige un
minimo global de "
            + MIN_ESTUDIANTES + " estudiantes.");
    }
}
```





**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



```
System.out.println(
    "    Actualmente solo hay " + listaEstudiantes.size() + " estudiantes
registrados.");
    } else {
        System.out.println("=====
=====");
        System.out.println("    REQUERIMIENTO 4: LISTADO DE
ESTUDIANTES    ");
        System.out.println("=====
=====");

        int totalAprobados = 0;
        int totalReprobados = 0;

        // Bucle optimizado de lectura secuencial rápida (For-each)
        for (Estudiante est : listaEstudiantes) {
            est.mostrarInformacion();
            if (est.determinarAprobacion()) {
                totalAprobados++;
            } else {
                totalReprobados++;
            }
        }

        System.out.println("\n=====
=====");
        System.out.println("    RESUMEN ESTADISTICO    ");
        System.out.println("=====
=====");

        System.out.println("    Total de estudiantes en el listado: " +
listaEstudiantes.size());
        System.out.println("    Alumnos Aprobados: " + totalAprobados);
        System.out.println("    Alumnos Reprobados: " + totalReprobados);
        System.out.println("=====
=====");
    }
    break;

case 3:
    System.out.println("-> Cerrando sesion. El sistema de calificaciones se ha
cerrado correctamente.");
    break;

default:
    System.out.println("-> [ERROR] Opcion fuera de rango. Por favor elija [1, 2 o
3].");
    break;
}
} while (opcion != 3);
}
```



## 9. PRUEBAS DE SOLUCIÓN DEL SISTEMA DESARROLLADO C++

- Menú de opciones dentro del programa en entorno Java

```
=====
      SISTEMA ACADEMICO DE CALIFICACIONES (JAVA - APE)
=====
1. Registrar Estudiantes (Mínimo global: 5)
2. Mostrar Listado Completo y Estadísticas de Notas
3. Salir del Sistema
-----
Seleccione una opción: █
```

Fig. 5. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado.

Fuente: Elaboración propia.

- Registro de 2 estudiantes dentro del sistema en el entorno Java

```
=====
      SISTEMA ACADEMICO DE CALIFICACIONES (JAVA - APE)
=====
1. Registrar Estudiantes (Mínimo global: 5)
2. Mostrar Listado Completo y Estadísticas de Notas
3. Salir del Sistema
-----
Seleccione una opción: 1

=== REGISTRO DE NUEVOS ESTUDIANTES ===
Estudiantes registrados actualmente: 0
Cual es la cantidad de estudiantes a registrar ahora?: 5

--- REGISTRANDO ESTUDIANTE #1 DE 5 ---
Ingrese el número de cédula: 0504341272
Ingrese los nombres: Gerardo Alexander
Ingrese los apellidos: Salazar Garcia
Ingrese la Nota del Parcial 1 (0.00 - 10.00): 10
Ingrese la Nota del Parcial 2 (0.00 - 10.00): 9.8
Ingrese la Nota del Parcial 3 (0.00 - 10.00): 3.4
-> [EXITO] Estudiante almacenado correctamente en memoria.
-> [EXITO] Respaldo guardado físicamente en 'registro_notas.csv'.

--- REGISTRANDO ESTUDIANTE #2 DE 5 ---
Ingrese el número de cédula: 0504341272
Ingrese los nombres: Toaquiña Nuñez
-> [ERROR] El campo solo debe contener letras y espacios (sin números ni símbolos).
Ingrese los nombres: Santiago Alexander
Ingrese los apellidos: Lema Tobar
Ingrese la Nota del Parcial 1 (0.00 - 10.00): 10
Ingrese la Nota del Parcial 2 (0.00 - 10.00): 1.2
Ingrese la Nota del Parcial 3 (0.00 - 10.00): 3.4
-> [EXITO] Estudiante almacenado correctamente en memoria.
-> [EXITO] Respaldo guardado físicamente en 'registro_notas.csv'.
```

Fig. 6. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado.

Fuente: Elaboración propia.



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO 2026 – JULIO 2026**



- **Reporte estadístico dentro del sistema desarrollado en Java**

```
=====
                        REQUERIMIENTO 4: LISTADO DE ESTUDIANTES
=====
Cedula: 0504341272
Alumno: Salazar Garcia Gerardo Alexander
Nota 1: 10,00 | Nota 2: 9,80 | Nota 3: 3,40
Promedio: 7,73
Estado: [APROBADO]
-----
Cedula: 0504341272
Alumno: Lema Tobar Santiago Alexander
Nota 1: 10,00 | Nota 2: 1,20 | Nota 3: 3,40
Promedio: 4,87
Estado: [REPROBADO]
-----
Cedula: 0504341272
Alumno: Pilco Lema Alex Mario
Nota 1: 3,40 | Nota 2: 3,60 | Nota 3: 7,80
Promedio: 4,93
Estado: [REPROBADO]
-----
Cedula: 0504341272
Alumno: Lema Atencia Antonio Mario
Nota 1: 10,00 | Nota 2: 3,40 | Nota 3: 2,30
Promedio: 5,23
Estado: [REPROBADO]
-----
Cedula: 0504341272
Alumno: Saltos Monar Mario Alex
Nota 1: 10,00 | Nota 2: 2,40 | Nota 3: 5,00
Promedio: 5,80
Estado: [REPROBADO]
=====

=====
                        RESUMEN ESTADISTICO
=====
Total de estudiantes en el listado: 5
Alumnos Aprobados: 1
Alumnos Reprobados: 4
=====

=====
SISTEMA ACADEMICO DE CALIFICACIONES (JAVA - APE)
=====
1. Registrar Estudiantes (Minimo global: 5)
2. Mostrar Listado Completo y Estadísticas de Notas
3. Salir del Sistema
=====
Seleccione una opcion: █
```

*Fig. 7. Reporte estructurado de estudiantes registrados con persistencia de datos en formato de texto plano.*

*Fuente: Elaboración propia.*